Field Observation Agent — Technical Design

Overview

The Field Observation Agent is a full-stack AI application that helps NGO field workers turn raw, unstructured field notes about at-risk children into structured, prioritized action plans. It uses a **ReAct (Reasoning + Acting)** agent loop powered by the Claude API, with a React frontend that streams the agent's reasoning in real time.

---

1. End-to-End Request Flow

Step 0 — User selects an observation and clicks "Analyze Observation"

The frontend (`ObservationInput.jsx`) calls `handleSubmit(observationText)` in `App.jsx` . This fires a `POST /api/analyze` request with the raw observation text in the body.

```
User clicks "Analyze Observation"
        │
        ▼
POST /api/analyze
{ "observation": "visited rampur today. priya hasnt been in school..." }
```

Step 1 — SSE connection is established

`agentRoutes.js` receives the request and immediately sets SSE response headers:

```
Content-Type: text/event-stream
Cache-Control: no-cache
Connection: keep-alive
X-Accel-Buffering: no
```

The HTTP connection stays open. From this point on, the backend pushes named events down the wire as they occur. The frontend reads them via the **Fetch Streams API** (`response.body.getReader()`), not the native `EventSource` API — this is intentional because `EventSource` does not support POST bodies.

Step 2 — ReAct loop begins (`reactLoop.js`)

`runReActLoop(observationText, emit)` is called. It initialises a conversation history array with a single user message:

```
messages = [
  {
    role: "user",
    content: "Field worker observation:\n\n\"...\"\n\nReason through this step by step."
  }
]
```

The loop runs for up to **6 iterations** (`MAX_ITERATIONS = 6`). Each iteration is one full LLM call.

Step 3 — LLM call (streaming) → Thought + Action

`streamCompletion(messages, SYSTEM_PROMPT, onToken)` is called via `claudeProvider.js`. The system prompt instructs the model to:

1. Output only plain text (no markdown).
2. Follow the strict `Thought:` / `Action:` / `Final Answer:` format.
3. Always begin by calling `get_beneficiary_history`, then `check_scheme_eligibility`, then `find_community_patterns`.

As tokens arrive from the Claude streaming API, two things happen simultaneously:

- Each token is buffered in `iterationBuffer` (server-side accumulation).
- Each token is emitted as an SSE `token` event to the frontend.

```
event: token
data: {"content": "The field worker mentions Raju from"}

event: token
data: {"content": " Shivpur. I need to look up his record."}
```

The frontend receives these and appends them to `tokenBuffer`, displaying them live in the "Thinking..." panel with a blinking cursor — the user sees the model "thinking" in real time.

A full iteration's LLM output looks like:

```
Thought: The observation mentions Raju from Shivpur working at a brick kiln.
I need to look up his record to get his beneficiary ID and attendance history.
Action: get_beneficiary_history({"query": "Raju Shivpur"})
```

Step 4 — Output parsing

After the stream completes, `iterationBuffer` holds the full text. Three parsers run:

`stripMarkdown(text)` — removes `**` and `*` in case the model adds bold formatting despite instructions.

`parseThought(text)` — regex `/Thought:\s*([\s\S]*?)(?=Action:|Final Answer:|$)/` extracts the reasoning text between `Thought:` and the next keyword.

`parseAction(text)` — regex `/Action:\s*(\w+)\(\s*(\{[\s\S]*?\})\s*\)/` extracts:

- Tool name (e.g. `get_beneficiary_history`)
- JSON params object (e.g. `{"query": "Raju Shivpur"}`)

`parseFinalAnswer(text)` — regex `/Final Answer:\s*([\s\S]+)/` detects if this iteration is the final one.

Step 5 — Discrete SSE events are emitted

After parsing, the backend emits structured events:

```
event: thought
data: {"content": "The observation mentions Raju from Shivpur...", "iteration": 1}

event: action
data: {"tool": "get_beneficiary_history", "params": {"query": "Raju Shivpur"}, "iteration": 1}
```

The frontend receives `thought` → clears the live streaming buffer and adds a completed blue "Thought" card to the reasoning chain. Then it adds an amber "Action" card showing the tool call.

Step 6 — Tool execution

`executeTool(toolName, params)` is called synchronously. Each tool reads the synthetic JSON data files at call time (with in-memory caching after the first read):

Tool	Data Source	Returns
`get_beneficiary_history`	`beneficiaries.json`	Full child profile, attendance history, computed risk level and trend
`check_scheme_eligibility`	`beneficiaries.json` + `intervention-knowledge.json`	Which schemes the child is enrolled in, eligible for but missing, and not eligible for
`find_community_patterns`	`intervention-knowledge.json`	Village-level risk patterns, attendance stats, past interventions, seasonal alerts

The result is serialised to JSON and:

1. Emitted as an SSE `result` event.
2. Injected into the conversation history as a new `user` turn: `"Result:\n{...}\n\nContinue your reasoning."` — this is the standard ReAct pattern of treating tool results as external observations.

```
event: result
data: {"tool": "get_beneficiary_history", "content": "{\"found\": true, ...}", "iteration": 1}
```

The frontend receives `result` → adds a green "Result" card with a summarised view (not raw JSON). It also sets `expectingNewThought = true`, which re-enables the live token streaming display for the next iteration.

Step 7 — Loop continues (iterations 2 and 3)

The conversation history now has:

```
messages = [
  { role: "user",      content: "Field worker observation: ..." },
  { role: "assistant", content: "Thought: ...\nAction: get_beneficiary_history(...)" },
  { role: "user",      content: "Result:\n{beneficiary data}\n\nContinue your reasoning." }
]
```

The model issues the next LLM call with this full history. It now knows the beneficiary ID from the first result and typically calls `check_scheme_eligibility` next, then `find_community_patterns`. Each iteration repeats Steps 3–6.

Step 8 — Final Answer

On the last iteration (typically iteration 3 or 4), the model outputs:

```
Thought: I now have Raju's full profile, his scheme eligibility gaps, and the
community context. The CWC referral is already pending but the Labour Department
was never engaged. This is a high-risk child labor case with debt bondage.
```

```
Final Answer: Raju Kumar (B002) is a 12-year-old SC category child from Shivpur
trapped in debt bondage at a brick kiln. His attendance has declined to near-zero
across 2024 and he is currently absent for 3 months. A CWC referral is pending
but the Labour Department has never been engaged, which is the critical missing
intervention. Immediate action is needed under the Child Labor Act.
```

`parseFinalAnswer()` detects this. The loop emits:

```
event: final
data: {"content": "Raju Kumar (B002) is a 12-year-old SC category child..."}
```

The frontend adds a purple "Agent Assessment" card and stops the live streaming display.

Step 9 — Action plan generation (`actionPlan.js`)

A **second, separate LLM call** is made — non-streaming, using `complete()`. This call:

- Takes only the **assistant turns** from the conversation history (the model's own reasoning text — compactly summarised), discarding the large raw tool result turns to preserve output token budget.
- Appends the Final Answer as context.
- Uses a separate system prompt instructing the model to output **only valid JSON** matching a strict schema with five sections: `immediate`, `mediumTerm`, `escalation`, `schemesToEnroll`, `successIndicators`.

```
event: status
data: {"message": "Generating action plan..."}
```

The raw response is regex-extracted (handles both bare JSON and markdown code fences), then `JSON.parse()`d. The parsed object is sent as:

```
event: plan
data: {
  "childName": "Raju Kumar",
  "village": "Shivpur",
  "riskLevel": "high",
  "primaryIssues": ["child labor", "debt bondage"],
  "immediate": [...],
  "mediumTerm": [...],
  "escalation": [...],
  "schemesToEnroll": [...],
  "successIndicators": [...]
}
```

Step 10 — Stream closed

```
event: done
data: {}
```

The frontend sets `isLoading = false`. The SSE connection closes.

---

2. Technical Design

Architecture Diagram

```
┌──────────────────────────────────────────────────────────────┐
│                       FRONTEND (React + Vite)                 │
│                                                               │
│  ObservationInput  ──POST /api/analyze──►                     │
│                                                               │
│  App.jsx                                                      │
│  ├── SSE reader (Fetch Streams API)                           │
│  ├── token events  ──► streamingText state ──► ReasoningChain  │
│  ├── thought/action/result events ──► steps[] ──► ReasoningChain│
│  ├── final event   ──► steps[] ──► ReasoningChain             │
│  └── plan event    ──► actionPlan state ──► ActionPlan        │
└──────────────────────────────────────────────────────────────┘
                            │ SSE stream
                            ▼
┌──────────────────────────────────────────────────────────────┐
│                   BACKEND (Node.js / Express)                 │
│                                                               │
│  agentRoutes.js                                               │
│  └── POST /api/analyze                                        │
│       │                                                       │
│       ├── runReActLoop()  ──────────────────────────────┐     │
│       │    │                                            │     │
│       │    ├── streamCompletion() ── Claude API (streaming) │  │
│       │    │        token by token ──► emit("token")       │  │
│       │    │                                            │     │
│       │    ├── parseThought / parseAction / parseFinalAnswer│  │
│       │    │        ──► emit("thought"), emit("action")    │  │
│       │    │                                            │     │
│       │    ├── executeTool()                            │     │
│       │    │   ├── getBeneficiaryHistory (beneficiaries.json)│ │
│       │    │   ├── checkSchemeEligibility (knowledge.json)  │  │
│       │    │   └── findCommunityPatterns (knowledge.json)   │  │
│       │    │        ──► emit("result")                   │     │
│       │    │                                            │     │
│       │    └── emit("final") when done ◄──────────────┘    │
│       │                                                       │
│       └── generateActionPlan()                                │
│            └── complete() ── Claude API (non-streaming)       │
│                 ──► emit("plan")                              │
│                                                               │
└──────────────────────────────────────────────────────────────┘
```

Component Responsibilities

Backend

File	Responsibility

`server.js`	Express app setup, CORS, port binding
`agentRoutes.js`	Route handlers, SSE lifecycle, orchestrates ReAct loop + action plan
`reactLoop.js`	ReAct loop: streaming LLM calls, output parsing, tool execution, conversation history management
`actionPlan.js`	Second LLM call with condensed history, JSON extraction, schema validation
`claudeProvider.js`	Claude SDK abstraction — `streamCompletion()` and `complete()`. Swap this file for Kaapi compatibility.
`tools/index.js`	Tool registry and `executeTool()` dispatcher
`tools/getBeneficiaryHistory.js`	Fuzzy search over beneficiaries by name, ID, or village
`tools/checkSchemeEligibility.js`	Rule-based eligibility evaluation across 12 government schemes
`tools/findCommunityPatterns.js`	Village-level pattern lookup + intervention playbook retrieval

Frontend

File	Responsibility
`App.jsx`	SSE consumer, state machine, coordinates all child components
`ObservationInput.jsx`	Textarea input + example observation loader
`ReasoningChain.jsx`	Renders streaming tokens live, then discrete thought/action/result/final cards
`ActionPlan.jsx`	Renders the structured plan in tiered cards (immediate / medium-term / escalation)

SSE Event Protocol

All events follow the SSE format: `event: <type>\ndata: <JSON>\n\n`

Event	When	Payload
`token`	Each LLM token during a ReAct step	`{ content: string }`
`thought`	After a complete Thought is parsed	`{ content: string, iteration: number }`
`action`	Tool call identified	`{ tool: string, params: object, iteration: number }`
`result`	Tool execution complete	`{ tool: string, content: string, iteration: number }`
`final`	ReAct loop concludes	`{ content: string }`
`status`	Informational	`{ message: string }`
`plan`	Action plan ready	Full plan object (see schema below)
`error`	Any failure	`{ message: string }`
`done`	Stream closed	`{}`

Action Plan JSON Schema

```
{
  "childName": "string",
  "village": "string",
  "riskLevel": "high | medium | low",
  "primaryIssues": ["string"],
  "immediate": [
    { "action": "string", "owner": "string", "deadline": "string", "note": "string" }
  ],
  "mediumTerm": [
    { "action": "string", "owner": "string", "deadline": "string", "note": "string" }
  ],
  "escalation": [
    { "trigger": "string", "action": "string", "contact": "string" }
  ],
  "schemesToEnroll": ["string"],
  "successIndicators": ["string"]
}
```

Data Layer

No database. Three JSON files loaded at startup with in-memory caching:

File	Contents
`beneficiaries.json`	20 child profiles with attendance history (quarterly), family background, risk factors, intervention history
`observations.json`	10 sample field observations (raw, messy text)
`intervention-knowledge.json`	12 government schemes with eligibility rules, 10 village community profiles, intervention playbooks per risk type

---

Provider Abstraction

`claudeProvider.js` exposes two functions and imports nothing else from the SDK:

```
streamCompletion(messages, systemPrompt, onToken) → Promise<string>
complete(messages, systemPrompt)                  → Promise<string>
```

To swap Claude for Kaapi or any other provider: replace only this file. The ReAct loop, action plan generator, routes, and tools are entirely provider-agnostic.

ReAct Loop — Conversation History Structure

After three tool calls, the `messages` array looks like this before the final LLM call:

```
[
  { role: "user",      content: "Field worker observation: ..." },
```

```
  { role: "assistant", content: "Thought: ...\nAction: get_beneficiary_history({...})" },
  { role: "user",      content: "Result:\n{beneficiary JSON}\n\nContinue your reasoning." },

  { role: "assistant", content: "Thought: ...\nAction: check_scheme_eligibility({...})" },
  { role: "user",      content: "Result:\n{scheme JSON}\n\nContinue your reasoning." },

  { role: "assistant", content: "Thought: ...\nAction: find_community_patterns({...})" },
  { role: "user",      content: "Result:\n{community JSON}\n\nContinue your reasoning." }
]
```

Tool results are injected as `user` turns — this is the standard multi-turn injection pattern for models that don't natively support tool-use. The model sees the result as an "observation from the environment" and continues reasoning.

Output Parsing Robustness

The model may produce markdown despite instructions. All parsers call `stripMarkdown()` first:

```
"**Thought:** ..." → "Thought: ..."
"**Action:** ..."  → "Action: ..."
```

The `parseAction` regex handles:

- Params on a single line: `Action: tool({"key": "val"})`
- Params spanning multiple lines (non-greedy `[\s\S]*?`)
- Whitespace between tool name and opening paren

If neither an `Action` nor a `Final Answer` is found, the loop terminates with an error event and logs the raw output server-side.

Token Budget Management

Two constraints prevent truncation:

ReAct steps — `streamCompletion` uses `MAX_TOKENS = 1024`. Each iteration outputs one Thought + one Action, which is well within this budget.

Action plan — `complete` uses `MAX_TOKENS = 8192` (the model's output ceiling). The conversation history passed to this call is condensed: only the assistant reasoning turns are included (the raw tool result JSONs, which can be 500–1500 tokens each, are stripped). The action plan prompt also caps each array to 3–4 items.

Responsible AI Considerations

- **Grounded output** — The agent can only reason about beneficiaries present in the JSON dataset. It cannot hallucinate children.
- **Cited evidence** — The system prompt requires the agent to cite specific numbers (attendance %, scheme names, risk factors) from tool results.
- **Legal accuracy** — The intervention knowledge base references real Indian government schemes (KGBV, CWC, PCMA, Child Labor Act) and the correct implementing bodies (DCPO, Block Education Officer, Labor Department).
- **Escalation path** — Every action plan includes an escalation section with legal levers (FIR, CWC protective order, Childline 1098) for the most severe scenarios.

- **Field worker language** — The action plan prompt explicitly requires simple language, and the UI presents plans in card form — not walls of text — for workers with limited digital literacy.

Running Locally

```
# 1. Add API key
echo "ANTHROPIC_API_KEY=sk-ant-..." >> backend/.env

# 2. Start backend (port 3001)
cd backend && npm run dev

# 3. Start frontend (port 5173)
cd frontend && npm run dev
```

Open http://localhost:5173, select any example observation, and click Analyze Observation.